

Universitatea
Transilvania
din Braşov

FACULTATEA DE INGINERIE ELECTRICĂ
ŞI ŞTIINŢA CALCULATOARELOR

PROIECT DE DIPLOMĂ

Conducători ştiinţifici:

conf. dr. ing. Danciu Gabriel Mihail

şef lucr. dr. ing. BOGDAN Ioana Corina

Absolvent:

Gîrbacea Marian Ionuţ

Braşov, 2026

Departamentul de Electronică și Calculatoare
Programul de studii: Calculatoare

GÎRBACEA Marian Ionuț

Sistem interactiv de antrenare VR în mediul industrial cu roboți

Conducători științifici:
conf. dr. ing. DANCIU Gabriel Mihail
șef lucr. dr. ing. BOGDAN Ioana Corina

Brașov, 2026

Cuprins

Lista de figuri, tabele și coduri sursă	3
Lista de acronime	4
1 Context și motivație	4
1.1 Context	4
1.2 Motivație	4
2 Obiectivele lucrării	6
3 Fundamente teoretice	7
3.1 Digital Twin	7
3.2 Cinematica inversă	7
3.3 Realitatea virtuală	7
3.4 MQTT	8
3.5 URDF	9
3.6 Roboții industriali	10
4 Tehnologii și aplicații folosite	12
4.1 Unity	12
4.1.1 "Unity XR Interaction Toolkit"	12
4.1.2 "Articulation Body"	13
4.1.3 "RayCast"	13
4.1.4 "RigidBody"	14
4.1.5 "Unity URDF Importer"	14
4.2 Fusion 360	14
5 Echipamente folosite	15
5.1 MaxArm	15
5.2 Oculus Quest 3S	17
6 Implementări	18
6.1 Realizarea modelului 3D al brațului robotic	18
6.2 Mediul din Unity	23
6.3 Importarea în Unity	24
6.4 Setarea proprietăților	25
6.4.1 Compensări cinematice	26
6.5 Sistemul de control al brațului în Unity	28
6.5.1 Metoda de control bazată pe control direct	28
6.5.2 Metoda de control bazată pe cinematică in- versă	29

6.6	Resetarea robotului	32
6.7	Manevrarea obiectelor	32
6.8	Sistemul de antrenare pentru operatorul uman . . .	34
6.8.1	Lista cu controale	34
6.8.2	Lista cu instrucțiuni de bază	34
7	Implementări	18
7.1	Realizarea modelului 3D al brațului robotic	18
7.2	Mediul din Unity	23
7.3	Importarea în Unity	24
7.4	Setarea proprietăților	25
7.4.1	Compensări cinematice	26
7.5	Sistemul de control al brațului în Unity	28
7.5.1	Metoda de control bazată pe control direct	28
7.5.2	Metoda de control bazată pe cinematică inversă	29
7.6	Resetarea robotului	32
7.7	Manevrarea obiectelor	32
7.8	Sistemul de antrenare pentru operatorul uman . . .	34
7.8.1	Lista cu controale	34
7.8.2	Lista cu instrucțiuni de bază	34
	Bibliografie momentană	37

Lista de figuri, tabele și coduri sursă

FIGURI

1	(Brațul Robotic MaxArm)	16
2	(Quest3S)	17
3	(Model 3D baza brațului)	18
4	(Model 3D bază secundară)	19
5	(Model 3D segment vertical principal)	19
6	(Model 3D segment orizontal principal)	19
7	(Model 3D segment lateral de urcare-coborâre)	20
8	(Model 3D tijă susținere 1)	20
9	(Model 3D tijă ajutătoare verticală 2)	20
10	(Model 3D tijă ajutătoare orizontală 1)	21
11	(Model 3D tijă ajutătoare orizontală 2)	21
12	(Model 3D suportul pompei de sucțiune)	21
13	(Model 3D ansamblul pentru ventuza de sucțiune)	22
14	(Model 3D)	23
15	(Structura ierarhica Unity)	24
16	(Model 3D baza brațului)	18
17	(Model 3D bază secundară)	19
18	(Model 3D segment vertical principal)	19
19	(Model 3D segment orizontal principal)	19
20	(Model 3D segment lateral de urcare-coborâre)	20
21	(Model 3D tijă susținere 1)	20
22	(Model 3D tijă ajutătoare verticală 2)	20
23	(Model 3D tijă ajutătoare orizontală 1)	21
24	(Model 3D tijă ajutătoare orizontală 2)	21
25	(Model 3D suportul pompei de sucțiune)	21
26	(Model 3D ansamblul pentru ventuza de sucțiune)	22
27	(Model 3D)	23
28	(Structura ierarhica Unity)	24

TABELE

CODURI SURSĂ

Lista de acronime

2D - 2 Dimensiuni;
3D - 3 Dimensiuni;
A - Amperi;
AR - Augumented Reality;
CAD - Computer Aided Design;
CAE - Computer Aided Engineering;
cm - centimetrii
; DC- Direct Current;
Kg - Kilograms;
CAM - Computer Aided Manufacturing;
mA - miliAmperi
mm-milimetrii;
PPD - Pixel Per Degree;
PPI - Pixel Per Inch;
RGB - Red Green Blue;
HMD - Head-Mounted Display;
HZ - Hertz
; IoT - Internet of Things;
MQTT - Message Queuing Telemetry Transport;
OASIS - Organization of Advancement of Structured Information Standard;
SCARA - Selective Compliance Articulated Robot Arm;
TMP - TextMeshPro;
UI - User Interface;
URDF - Unified Robotics Description Format;
USB - Universal Serial Bus;
XML - Extensible Markup Language;
XR - Extended Reality;
V - Voltage;
VR - Virtual Reality;
WIFI - Wireless Fidelity;

1 Context și motivație

1.1 Context

În societatea contemporană majoritatea domeniilor cum sunt cele de producție, farmaceutică până la apărare sau alimentar prezența echipamentelor de tip roboți sau brațe robotice a devenit tot mai răspândită, unde se integrează expertiza umane cu cea a asistenței din partea roboților. Astfel a apărut o nevoie pentru testarea și găsirea mai rapidă a erorilor și defectelor unor astfel de echipamente din partea producătorilor și antrenarea directă pe aceștia poate duce la defectarea lor sau chiar la accidentări, ducând la costuri suplimentare atât din punct de vedere al companiei care a achiziționat robotul cât și din partea producătorului robotului.

O soluție pentru testarea și găsirea eventualelor erori și defecte o reprezintă tehnologia "digital twin". Pentru partea de antrenare a operatorului uman se poate folosi realitatea virtuală pentru a implementa un mediu de antrenament ce implementează un "digital twin", mediul cuprinzând majoritatea scenariilor în care operatorul uman și brațul robotic pot să fie supuse în activitățile la care sunt implicate.

O interacțiune directă dintre robotul real și "digital twin" poate fi utilă în momentul în care este necesară controlul direct al brațului real sau al robotului în medii periculoase cum sunt cele manevrarea elementelor radioactive sau a fiolelor din laboratoare.

Lucrarea își propune realizarea unui astfel de mediu de antrenament în realitatea virtuală folosind implementarea unui "digital twin" pentru un braț robotic cu 4 grade de libertate ce permite ridicarea și mutarea de obiecte folosind o pompă. De asemenea acest sistem urmărește ca procesul de antrenare a utilizatorilor umani, care nu au interacționat niciodată sau rar cu un astfel de sistem să fie cât mai simplă, intuitivă astfel încât timpul petrecut în antrenare să fie cât mai mic și nivelul de pregătire dobândit să fie cât mai mare.

1.2 Motivație

Motivația principală din spatele realizării constă în creșterea în popularitate a tehnologiei de "digital twin" în domenii precum: medical, de producție domeniul energetic [12]. Pe partea de producție aceștia pot ajuta operatorul să înțeleagă condițiile asset-ului și să realizeze planuri predictive pentru mentenanță. Producătorii folosind "digital twins" pentru planificarea zonei de producție, pot să simuleze impactul asupra liniei de producție, ajustând și optimizând pentru cel mai bun flux de lucru. În domeniul auto, producătorii deja folosesc "digital twins" pentru a prototipa și simula modele noi de vehicule. Aceștia pot să ofere posibilitatea pentru vehicule

pe bază de software și mașini autonome [16]. Pentru domeniul energetic, "digital twins" pot fi folosiți în rețele electrice și tehnologia microgrid, pentru a ajuta operatorii în optimizarea și distribuția de energie. Flexibilitatea acestei tehnologii le oferă operatorilor noi unelte pentru a răspunde într-un mod proactiv la deranjamente, să simuleze situații de urgență și în planuirea integrării surselor de energie regenerabilă eficientă din punct de vedere al costului. În medicină, le oferă furnizorilor de servicii medicale posibilitatea îmbunătățirii uneltelor de monitorizarea a pacienților. Prin combinarea datelor de la dispozitive multiple destinate pacienților într-un "digital twin", furnizorii pot să obțină un o vedere mai mare de ansamblu asupra sănătății pacientului. De asemenea, pot ajuta administratorii să monitorizeze rata de ocupare și îngrijorările legate de siguranță sau chiar să monitorizeze calitatea aerului pentru a ajuta în prevenirea răspândirii patogenilor [16].

Un alt domeniu în care această tehnologie poate fi folosită este cea a telecomunicațiilor. Aceasta le poate oferi furnizorilor din telecom să obțină o imagine de ansamblu mai bună asupra infrastructurii rețelei, a stării asset-ului și să ofere posibilitatea de a se realiza mentenanță predictivă pentru minimizarea momentelor de picare a rețelei. Pe lângă furnizorii îmi mai pot folosi în simularea fluctuării cererilor din rețea pentru teste de stres sau planificarea alternativelor fără necesitate de schimbări costisitoare în lumea reală [16].

Un avantaj principal al acestui proiect constă în folosirea tehnologie de realitate virtuală, care reprezintă o soluție modernă care permite simularea și controlul acestor roboți, oferindu-le utilizatorilor posibilitatea de a învăța cum să-i controleze, limitele lor și scenariile de utilizare fără a fi necesar interacțiunea fizică cu cei reali. Un sistem în realitatea virtuală poate reduce costurile, crește eficiența în antrenare și de a îmbunătăți nivelul de pregătire fără a afecta productivitatea reală.

Un alt aspect important al proiectului constă în implementarea a 2 metode de control al brațului din VR. Această metodă de implementare a metodei de control prezintă un avantaj semnificativ deoarece oferă utilizatorului posibilitatea de control al robotului într-un mod simplu și ușor de înțeles, cât și producătorului să testeze rapid implementări și să rezolve eventualele probleme apărute.

2 Obiectivele lucrării

Principalele obiective stabilite pentru realizarea lucrării:

- Dezvoltarea unui sistem de antrenament care să ofere un minim de pregătire asupra controlului robotului
- Realizarea modelului 3D al brațului din realitate
- Importarea modelului 3D al brațului în mediul VR din Unity
- Aplicarea proprietăților și constrângerilor robotului care sunt necesare pentru mișcarea acestuia
- Realizarea primului mod de mișcare al robotului baza pe asociere directă între butoanele de la manete și motoare
- Realizarea celui de al 2-lea mod de control al robotului bazat pe cinematică inversă
- Aplicarea valorilor citite din Unity asupra robotului din lumea reală pentru a putea fi controlat
- Primirea feedback-ului de la robotul real legat de valorile pe care le-a recepționat

3 Fundamente teoretice

3.1 Digital Twin

Un "digital twin" este o reprezentare virtuală a unui obiect fizic sau sistem ce folosește date din timp real să reflecte cu acuratețe comportamentul geamănului din lumea reală, performanțe și condiții. "Digital twins" permit monitorizate continuă, simulare și analiză a unui obiect, sistem sau produs pe durata timpului său de viață, de la proiectare și producție, până la mentenanță și dezafectare. Aceștia pot să reprezinte orice obiect într-un mod virtual, de la clădiri și poduri, până la mașini, avioane, artefacte antice și chiar întreaga planetă [17].

De asemenea, prin "digital twins" pot să se modeleze sisteme complexe cum sunt: traficul, evenimente meteorologice, planuri medicale de tratament, procese și operațiuni din fabrici sau în medii experimentale, și chiar pot să fie bazați fie pe oameni reali, fie imaginari cu toate trăsăturile necesare cum sunt: tonul vocii, înfățișare și personalitate [17].

3.2 Cinematica inversă

Cinematica reprezintă studiul mișcării fără să se considere cauza mișcării, cum sunt forțele și cuplurile. Cinematica inversă reprezintă folosirea ecuațiilor cinematice să determine mișcarea unui robot pentru a ajunge la destinație. De exemplu, pentru a putea apuca un obiect, un braț robotic folosit în procese de fabricație necesită mișcări precise de la poziția inițială către poziția necesară. Capătul de prindere al robotului reprezintă efectorul final. Configurația robotului reprezintă o listă a pozițiilor de "Joint-uri" care pot fi acționate atât timp cât nu depășesc constrângerile robotului.

Pornind de la poziția la care efectorul final trebuie să ajungă, cinematica inversă poate să determine o configurație a "Joint-urilor" astfel încât efectorul final să poată să se miște către țintă [19].

3.3 Realitatea virtuală

Realitatea virtuală reprezintă un termen care descrie un mediu 3D generat de un calculator care poate fi explorat și se poate interacționa de către un utilizator uman. Acel utilizator devine o parte a mediului virtual sau devine imersiv cu mediul și cât timp se află în mediu, poate să manipuleze obiectele sau să performeze o serie de acțiuni.

Realitatea virtuală este implementată folosind sisteme care sunt destinate pentru realizarea mediului, cum sunt căștile, benzile de mers multidirecționale și mănuși speciale. Aceste sisteme sunt folosite să stimuleze simțurile simultan pentru a crea o iluzie a realității

Această tehnologie poate fi împărțită în 5 tehnologii principale:

- Realitatea virtuală non-imersivă se referă la utilizarea unor serii de ecrane care înconjoară utilizatorul pentru a fi prezentată informația virtuală[4].
- Realitatea virtuală semi-imersivă în care componentele digitale se suprapun peste obiectele reale. Astfel aceste elemente virtuale pot fi folosite într-un mod similar cu cele reale [22].
- Realitatea virtuală imersivă, care a fost folosită în acest proiect se folosește de folosirea unui HMD pentru a urmări mișcările utilizatorului și să prezinte informația VR în funcție de mișcările utilizatorului[4].
- Realitatea augmentată care reprezintă un alt mod de a realiza realitate virtuală prin modul în care dezvoltatorii suprapun cele ambele realități [22].
- Realitate mixtă combină lumea fizică cu cea virtuală. Reprezintă un caz special de realitate augmentată. Această tehnologie permite vizualizarea de oameni și obiecte într-un context real [22].

3.4 MQTT

MQTT reprezintă un standard OASIS fiind un protocol de mesagerie folosit în IoT. Este proiectat ca să implementeze protocolul de tip publicare/abonare într-un mod în care să folosească cât mai puține resurse. Acest protocol este ideal pentru conectarea între dispozitive, ce sunt limitate din punct de vedere hardware. Comunicarea prin acest protocol este bidirecțională. [9].

Mesageria de tip publicare/abonare reprezintă un model de comunicare asincron care permite dezvoltatorilor să construiască aplicații funcționale și complexe în cloud. Pentru aceste sisteme cloud modelul de mesagerie oferă notificări instantanee când se întâmplă un eveniment în sistemele distribuite [20].

Acest model funcționează în felul următor:

- Mesajele reprezintă un set de date de comunicație care sunt trimise de la emițător la receptor. Mesajele pot fi de orice tip de dată [20]
- Topicul care acționează ca un canal intermediar între emițător și receptor. Gestionează o listă de receptori care sunt interesate în mesajele care au același topic [20]
- Abonații reprezintă destinatarul mesajului. Aceștia trebuie să se înregistreze sau să se aboneze la un topicul care îi interesează. Pot să performeze diferite funcții sau să facă ceva diferit cu mesajul în paralel [20]

- Publicatorii reprezintă componenta care trimite mesajele. Creează mesaje despre un topic și le trimite pe toate la toți abonații ai acelui topic. Interacțiunea dintre publicator și abonați reprezintă o relație de tipul unul la mulți. Publicatorul nu trebuie să știe cine folosește informația, iar abonații nu trebuie să cunoască proveniența mesajelor, transmisia fiind una de difuzare [20]

Modelul de mesagerie de tip publicare/abonare reprezintă următoarele avantaje:

- Eliminarea polling-ului. Topicurile de mesaje care permit transmiterea mesajelor instantaneu eliminând verificările periodice pe care consumatorii mesajului trebuie să le facă când apare informație nouă sau când este actualizată [20].
- Direcționare dinamică. Modelul publicare/abonat face ca descoperirea serviciilor mai ușor, mai natural, și mai greu susceptibilă la erori. În loc ca aplicația să întrețină un grup de abonați căruia să-i trimită mesajele, un publicator va posta mesajele către un topic specific, astfel încât abonații să se aboneze la mesajele cu topic-urile de interes [20].
- Simplificarea comunicației. Acest model simplifică procesul de comunicare prin eliminarea conexiunilor punct-la-punct care leagă o singură conexiune către un topic. Topicul va gestiona abonările, astfel încât să decidă ce mesaje sunt destinate cărui endpoint [20].

Aceste mesaje pot fi consumate de oricare număr de noduri, incluzând, filtre, sisteme de nivel înalt cum ar fi cele de ghidare [7].

3.5 URDF

URDF este un limbaj markup bazat pe XML, dezvoltat ca parte a ecosistemului ROS. Este folosit în a reprezenta modelul unui robot în raport cu: structura fizică, cinematica, proprietățile legate de inerție, aspectul fizual și geometria coliziunilor [21].

Un fișier definește un robot ca o colecție de legături (corpuri rigide), conectate prin articulații (care definește cum legăturile se pot mișca între ele). De asemenea poate specifica senzori, transmisii și materiale, permițând simularea și vizualizarea robotului în mediul 3D [21].

URDF servește ca o componentă esențială în controlul prin:

- Simulare: Modelele URDF sunt folosite pentru a încărca roboții în simulatoare cum este Gazebo, sau medii de dezvoltare 3D cum este Unity, unde dezvoltatorii testează algoritmi în medii de dezvoltare virtuale controlate înainte să fie încărcate pe hardware. URDF se asigură că

constrângerile fizice și reprezentările vizuale se aseamănă cu cele din lumea reală [21].

- **Cinematica și planificarea mișcărilor.** Este necesară pentru a ajuta la înțelegerea limitelor articulațiilor și ierarhiei dintre legături și coordonatele componentelor. Acestea permit planificarea mișcărilor cu acuratețe și calculului cinematicii inverse și directe [21].
- **Vizualizare și Debugging.** URDF este folosit pentru a randa vizualizări 3D ale robotului în timp real. Se pot suprapune datele de la senzori pe model, ceea ce face mai ușor înțelegerea comportamentului sistemului și debugging-ul [21].
- **Integrarea cu senzori.** URDF permite poziționarea cu precizie a cadrelor și a pozițiilor senzorilor pe robot. Modelarea acestora cu acuratețe este esențială pentru combinarea senzorilor și a navigației [21].

3.6 Roboții industriali

Roboții industriali sunt folosiți în procese de asamblare, curățare și celelalte activități din fabrici unde oamenii nu pot opera. Automatizarea fiecărui proces îmbunătățește eficiența și contribuie la reducerea lipsei forței de muncă și al costurilor [14].

Roboții industriali se împart în 6 tipuri:

- **Roboți liniari.** Aceștia sunt asemănători în aspect cu o macara de tip pod cu 3 axe de mișcare, are o capacitate de mutare de obiecte și acuratețe ridicată. Axele acestui robot sunt perpendiculare între ele și robotul se mișcă cu axele în linie dreaptă [1].
- **Roboți cilindrici.** Aceștia sunt roboți pe 3 axe cu zone de lucru sub formă de cilindri, 2 axe ale robotului se mișcă linear, iar cel de al 3 lea se mișcă circular [1].
- **Roboți sferici.** Aceștia au sistem de coordonate polare. Au 2 axe de rotație și o axă lineară.
- **Roboți SCARA.** Sunt roboți de mare precizie și viteză ce au 4 axe de mișcare. Încheieturile roboților se mișcă pe un plan orizontal împreună cu alte 3 axe, și o altă componentă se mișcă sus și jos împreună cu cea de a 4 a axă [1].
- **Roboți articulați.** Sunt roboți pe 6 axe compuse din încheieturi conectate secvențial, care copiază aspectul unei mâini umane. Cele 6 axe oferă robotului flexibilitate și le permite axelor să ajungă în zone în care ceilalți nu pot [1].

- Robot delta. Sunt roboți ce au 3 mâner atașate de o bază și care țin componenta de lucru a robotului paralelă cu baza. Aceștia pot avea 3, 4 și 6 axe de mișcare. Varianta cu 3 axe poate muta obiectele pe planuri paralele, și variantele cu un număr mai mare de axe sunt capabile să rotească obiecte și să manipuleze obiecte din părți diferite [1].

4 Tehnologii și aplicații folosite

4.1 Unity

Unity reprezintă o platformă interactivă de creare de conținut 3D, 2D pentru VR și AR. [15].

Acesta este un motor de creare de conținut în timp real pentru crearea de experiențe care sunt interactive într-un mod complet care este destinat atât utilizatorului final cât și pentru creatorii de conținut [15].

Termenul de timp real descrie rapiditatea cu care o imagine este randată sau afișată pe un ecran chiar și când aceasta este schimbată de către utilizator. Scopul programelor software de timp real este de a randa imaginile atât de rapid încât interacțiunea cu mediul virtual să se facă fără observarea acută a întârzierilor[15].

Unity are aplicații în următoarele domenii: media, divertisment, arhitectură inginerie, construcții, automotiv, transport și producție[15].

Unity în acest proiect a fost folosit pentru simplitatea cu care acest motor de creare a conținutului 3D integrează tehnologii de modelare de conținut 3D, cât și tehnologii de vizualizare și interacțiune cu roboții[15].

4.1.1 "Unity XR Interaction Toolkit"

"XR Interaction Toolkit" este un sistem de nivel înalt și bazat pe componente de interacțiune pentru crearea de aplicații VR și AR. Oferă un framework care poate fi folosit în dezvoltarea de interacțiuni de 3D și UI, acestea fiind disponibile din Unity Input Events. Centrul acestui sistem este bazat pe un interactor și componente interacționabile, și un gestionator de interacțiuni care leagă împreună aceste tipuri de componente. De asemenea conține componente care pot fi folosite pentru locomotie și desenare [23].

"XR Interaction ToolKit" conține următorul set de componente care suportă următoarele acțiuni de interacțiune:

- Compatibilitate cu o gamă înalte de platforme folosite pentru control: "Meta Quest, OpenXR, Windows Mixed Reality și altele" [23]
- Manipulare de bază a obiectelor din scenă [23]
- Feedback haptic prin manetele de XR [23]
- Feedback vizual pentru indicarea interacțiunilor active și posibile [23]
- Interacțiune utilă cu "XR Origin", ce reprezintă o componentă de cameră VR folosită în manevrarea staționară și în funcție de mărimea zonei a experiențelor VR [23]

4.1.2 "Articulation Body"

"Articulation body" reprezintă o componentă ce permite construirea de articulații fizice cum sunt brațele robotice, lanțuri cinematice cu "GameObjects" care sunt organizate într-un mod ierarhic de tipul părinte-copil sau arbore. Acestea ajută în obținerea de fizică comportamentală realistă în contextul simulărilor și aplicațiilor industriale [2].

Un "articulation body" permite definirea pentru un singur obiect proprietăți, care pot fi definite de alte componente asemănătoare cum sunt "RigidBody" sau "Regular Joint" într-o configurație clasică. Aceste proprietăți depind de tipul de poziția obiectului în ierarhie [2].

Acestea sunt:

- pentru obiectul rădăcină sau baza: masa, mutabilitatea, și dacă să folosească gravitația [2]
- pentru obiectele copil sau brațe: masa, dacă să folosească gravitația, tipul pe rotație al articulației, legarea de ancora părintelui, poziția și rotația ancorei, fricțiunea articulației, amortizarea liniară și unghiulară, gradele de libertate ale mișcării, limitele unghiulare superioare și inferioare, rigiditatea, amortizarea, limita forței, poziția unghiului și viteza de deplasare [2].

Articulațiile din "articulation body" sunt rezolvate folosind metoda "Featherstone" care lucrează folosind coordonate reduse prin: fiecare corp, copil sau componentă are coordonate relative față de părintele acestuia dar numai în jurul gradelor de libertate pe care le are setate. Acestea garantează că nu vor exista întinderi care nu sunt necesare [3].

4.1.3 "RayCast"

"RayCast" reprezintă o metodă comună de identificare al unui obiect în Unity, fiind necesară în selectarea unui obiect. Acesta funcționează prin emiterea unui "ray" dintr-un punct de origine dat, în direcția și distanța necesară. De asemenea pot fi folosit în ignorarea unui obiect dintr-un strat și să ignore sau nu obiecte ale căror coliziuni sunt folosite ca declanșatoare. Pentru a folosi "RayCast" este absolut necesar ca obiectul să aibă implementată componenta din Unity de coliziuni pentru a fi detectată [10].

4.1.4 "RigidBody"

Este o componentă care în momentul în care este aplicată pe un obiect, acel obiect va fi controlat prin motorul de fizică din Unity. Fără script-uri suplimentare, un obiect cu "RigidBody" va fi tras de gravitație și va reacționa cu obiectele din jur numai dacă au același tip de coliziuni. Aflându-se sub controlul motorului de fizică, este recomandat să se folosească funcția "FixedUpdate()", deoarece modificările de fizică sunt realizate în pași de timp care nu coincid cu actualizările de cadru [11].

4.1.5 "Unity URDF Importer"

"Unity URDF Importer" reprezintă un pachet pentru Unity ce permite importarea unui robot definit prin formatul URDF într-o scenă din Unity. Importatorul pasează un fișier URDF și îl importă în Unity folosind "articulation body" [13].

4.2 Fusion 360

Fusion 360 reprezintă o platformă care combină CAD, CAM, CAE, PCB, gestionarea datelor într-o singură și integrată platformă software în cloud [18].

Fusion este capabil să:

- De explorarea ale iterățiilor proiectului cu ușurință folosind unelte de modelare 3D [18]
- Să producă părți de mașini CNC la o calitate înaltă cu uneltele de CAM/CAM integrate [18]
- Să acceseze proiecte electronice întregi [18]
- Oferă unelte de simulare 3D pentru proiect [18]
- Se poate explora rezultate care sunt pregătite pentru producție cu proiect generativ [18]

5 Echipamente folosite

5.1 MaxArm

MaxArm este un brat robotic dezvoltat de compania Hiwonder. Este un brat robotic open-source care este controlat de un microcontroler ESP32. Poate fi folosit în scop educativ sau în proiecte avansate. Robotul este echipat cu servo motoare cu magistrală HTS-35H de înaltă calitate și o pompă de sucțiune [6]. Aceste servomotoare sunt controlate prin comenzi provenite de la portul serial, fiind necesar setarea parametrilor servovului și ID înainte să poată fi folosit. Acest servomotor folosește interfața UART pe "half-duplex", astfel încât terminalul pentru semnal să poată să transmită și să recepționeze semnale.

Brațul prezintă un mecanism articulat specific folosit în construcția de mașini, pentru care necesită utilizarea unor servomotoare cu ax unic, capabile de generarea unui cuplu puternic și să ofere o capacitate de poziționare de înaltă precizie, aceste motoare îndeplinind aceste condiții [5]

Specificații motoarelor HTS-35H [5]:

- Tensiune de lucru: 9-12V DC
- Viteză de rotație 0.18s/60° (DC 11.1V)
- Cuplu 35kg.cm (DC 11.1V)
- Cuplu static maxim 35kg.cm (DC 11.1V)
- Intervalul de rotație: (0 – 240)°
- Curent la mers în gol: 100mA
- Curent de blocaj: 3A
- Acuratețe motor: (0.2)°
- Interval de control al unghiurilor: 0-1000 corespunzând cu 0 – 240°
- Interval de ID-uri: 0-253
- Protecții: Evitarea stărilor de blocaj și supraîncălzire
- Parametrii de feedback: Temperatură, tensiune, poziție
- Mod de lucru: Mod de servo și în mod de motor cu rotație continuă
- Tip angrenaj: Metalic
- Greutate 64g

- Dimensiuni: 54.38*20.14*45.5 mm (lungime*lățime*înălțime)

Prezintă 4 grade de libertate: stânga-dreapta, sus-jos, față-spate, și rotația de obiecte. Prin aceste grade și tehnologia de cinematică inversă, acesta poate să execute o gamă largă de sarcini cum ar fi: apucare, sortare, transportare și stivuire de obiecte. Acesta suportă comunicare prin Bluetooth și WIFI și programare folosind Python prin Micro-Python și Arduino prin C/C++ [6].

Specificații [6]:

- Greutate: 1.3Kg
- Metal și placă din fibră de sticlă
- Grade de libertate: 4
- Alimentare: 12V 5A DC
- Controler: ESP32
- Metode de comunicație: WIFI și Bluetooth
- Motoare: HTS-35H și LFD-01M
- Dimensiuni: 158*160*260 mm



Figura 1: Brațul Robotic MaxArm

5.2 Oculus Quest 3S

Oculus Quest 3S sunt o pereche de ochelari de realitate mixtă de buget dezvoltată de compania Meta [8].

Specificații [8]:

- Procesor: Snapdragon XR2 Gen 2
- Camere: 2 camere RGB cu PPD de 18
- Stocare: 128GB/256GB
- DRAM: 8GB
- Rezoluție ecran: 1832x1920 pixeli/ochi cu 20 PPD și 773 PPI
- Rată de reîmprospătare: 72Hz, 90Hz, 120Hz
- Unghi de vizualizare: 96 de grade orizontal, 90 de grade vertical
- Optică: lentile fresnel
- Conectivitate: USB-c și WIFI
- Durată baterie: aproximativ 2.6 ore
- Timp de încărcare: aproximativ 1.9 ore pe 18Watts



Figura 2: Ochelari de realitate virtuală Oculus Quest3S

6 Implementări

6.1 Realizarea modelului 3D al brațului robotic

Pentru a putea implementa brațul robotic în Unity a fost necesară realizarea modelului 3D. Pentru a realiza modelul 3D a fost necesară împărțirea brațului robotic pe componente individuale și măsurarea acestora. Astfel modelul a fost împărțit în 11 părți, având următoarele dimensiuni aproximative din lipsa de instrumente de măsurat de precizie exprimate în milimetri:

- Baza principală robotului: 140 x 140 x 66

Aceasta reprezintă baza robotului de care este prinsă componenta de rotație stânga-dreapta pe motorul intern care mai conține și pompa prin care produce vacuumul pentru sucțiune

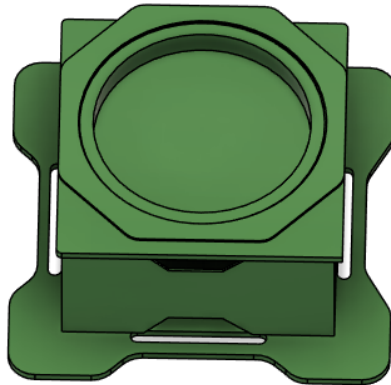


Figura 3: Modelul 3D al bazei brațului

- Baza secundară: 109 x 85 x 63

Este componenta care se rotește stânga-dreapta atașată de motorul din bază și de care sunt atașate cele 2 motoare de pe o parte și alta care-i permit brațului să deplaseze sus-jos față spate, și de care mai sunt atașate tijele de susținere verticală și brațul vertical principal.

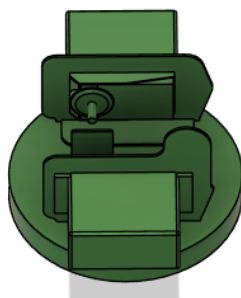


Figura 4: Modelul 3D al bazei secundare

■ Segment vertical principal: 20 x 15 x 150

Acesta este prins de motorul din stânga de pe zona rotativă prin care brațul se poate deplasa față-spate. Acesta funcționează și ca susținere pentru brațul orizontal principal.



Figura 5: Modelul 3D al segmentului vertical principal

■ Segment orizontal principal: 180 x 25 x 25

Este prinsă de segmentul vertical principal și tija ajutătoare verticală 1 și de aceasta se ocupă în principal de susținerea suportului pentru pompă.



Figura 6: Modelul 3D al segmentului orizontal principal

■ Segment lateral de urcare-coborâre: 67 x 2 x 20

Acesta este atașat la motorul drept de pe zona rotativă și prin intermediul acestuia se poate deplasa brațul sus-jos.

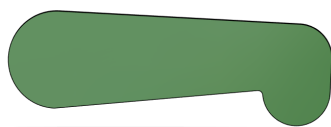


Figura 7: Modelul 3D al segmentului lateral de urcare-coborâre

■ Tijă ajutătoare verticală 1: 5 x 3 x 140

Este prinsă între segmentul lateral de urcare-coborâre și segmentul orizontal principal și are ca rol să ajute la urcarea sau coborârea segmentului orizontal principal și de asemenea la susținerea brațului când trebuie să se deplaseze prin segmentul vertical principal față-spate.



Figura 8: Modelul 3D al tije ajutătoare verticală 1

■ Tijă ajutătoare verticală 2: 15 x 5 x 144

Este prinsă între zona rotativă și tija ajutătoare orizontală 2 și are ca rol să ofere robotului o stabilitate mai bună atât când se deplasează față-spate cât și când stă nemișcat.



Figura 9: Modelul 3D al tije ajutătoare verticală 2

■ Tijă ajutătoare orizontală 1: 146 x 5 x 5

Este prinsă între tija ajutătoare orizontală 2 și suportul pentru pompa de sucțiune și are ca rol să ajute în susținerea și balansul suportului pentru pompă.



Figura 10: Modelul 3D al tije ajutoare orizontale 1

■ Tijă ajutătoare orizontală 2: 84 x 5 x 37

Este prinsă între tija ajutătoare 1, tija verticală de susținere 2 și segmentul vertical principal și are ca rol să susțină tija ajutătoare 1 să se deplaseze după segmentul orizontal principal și să ajute tija verticală de susținere 2 să se deplaseze după segmentul vertical principal.

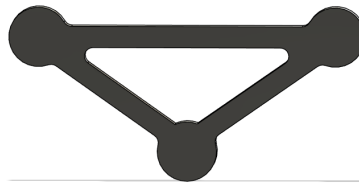


Figura 11: Modelul 3D al tije ajutoare orizontale 2

■ Suportul ansamblului pentru ventuza de sucțiune : 42 x 1 x 44 Este prinsă de segmentul orizontal principal și de tija orizontală ajutoare 2 și are ca rol să susțină ansamblul pentru ventuza de sucțiune cât și mini motorul.

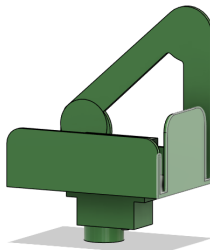


Figura 12: Modelul 3D al suportului pompei de sucțiune

■ Ansamblul pentru ventuza de sucțiune : 26 x 26 x 35

Acesta este prinsă de suport și este conectată printr-un furtun la pompa din interiorul bazei principale și este componenta prin care brațul robotic poate să manipuleze obiecte.

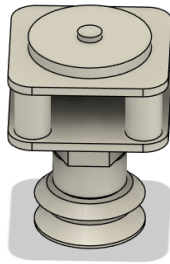


Figura 13: Modelul 3D al ansamblului pentru ventuza de sucțiune

După modelarea individuală a componentelor a urmat prinderea acestora între ele. Pentru a le putea uni s-a folosit "Joint" din Fusion. "Joint" permite prinderea componentelor între ele și stabilirea tipului și sensului/unghiului de rotație/deplasare. Principalul "joint" folosit pentru prindere și stabilire unghiului de rotație este cel de tip "revolute" care permite rotații de până la 360 de grade. Vizual aceste "joints" sunt reprezentate sub formă de cerc ce are prins un steguleț, steguleț ce reprezintă sensul pozitiv/negativ de rotație al componentei.

Prinderea între 2 componente în Fusion a fost realizată folosind numai un singur "joint" între componenta care trebuie prinsă și componenta de care se prinde.

Prinderea folosind un singur "joint" a fost necesară deoarece dacă se folosesc 2 "joints" pentru a se prinde o componentă între altele 2 apar conflicte între "joints" ceea ce ar face componenta prinsă să nu se mai miște, neștiind de care este controlată.

Acest lucru a mai fost necesar pentru a transforma dintr-o structură de paralelogram așa cum este robotul real, într-o structură de arbore. Structura de arbore este necesară pentru a putea genera într-un mod complet fișierul URDF al robotului, folosit pentru a-l putea importa în Unity.

Generarea fișierului URDF a fost realizată folosind un script de conversie care analizează modelul 3D de la bază la ultima componenta atașată și invers, și în funcție de "joints" care sunt prinse generează:

- partea de "link" care se ocupă de gestionarea părții vizuale ale componentelor
- partea de "joints" care se ocupă de partea de mișcare ale componentelor

Modelul final al robotului după ce toate componentele au fost prinse și unghiurile de rotație au fost setate:

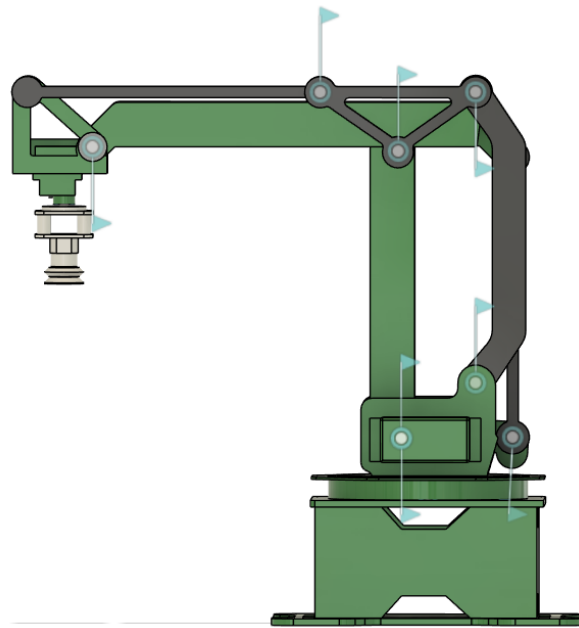


Figura 14: Modelul 3D al brațului MaxArm

După generarea fișierului URDF, acesta a fost importat în Unity. Însă până la importul în Unity a fost necesar un mediu de lucru cu care se poate interacționa în VR.

6.2 Mediul din Unity

Pentru mediul de lucru în VR s-a folosit ca bază un mediu demo oferit de cei de la Unity, care oferă un minim necesar pentru implementările ulterioare, acestea fiind: scripturi necesare pentru mișcarea și interacționarea cu mediul, elemente cu care se poate interacționa, un mini tutorial despre cum funcționează mediul, și alte elemente.

Fiind un mediu demo, acestuia i-sa adus câteva modificări necesare pentru al aduce la un stadiu în care se pot realiza implementările ulterioare: dezactivarea manetei drepte, și a unor butoane de pe maneta stângă, fiind ulterior folosite pentru a implementa sistemul de control al brațului, și re-maparea acestora, eliminarea unor elemente de decor.

6.3 Importarea în Unity

Importarea în Unity a fost realizată prin folosirea pachetului URDF care preia structura și ierarhia din fișierul URDF generat anterior, le analizează, o importează în Unity și generează fișierele de textură fiecărei componente.

Pe lângă import și texturi pachetul atribuie fiecărei componente un corp de tip "articulation", scripturile de definire al tipului de "joint", de control al "joints" și de parametri fizici. Problema cu script-ul este că deși generează fișierele de textură, acesta nu aplică automat texturile pe componente, aplicarea acestora realizându-se manual.

Structura ierarhică a componentelor:

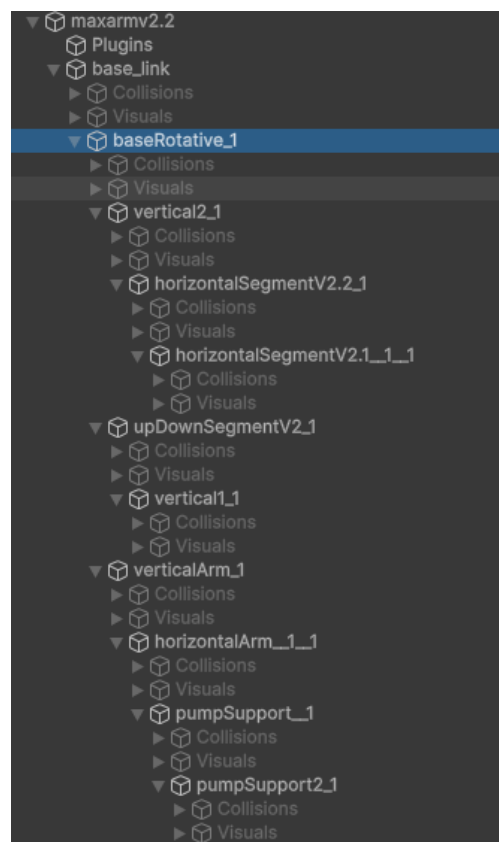


Figura 15: Structura ierarhica în Unity al brațului robotic

Având structura robotului și modelul 3D importat, texturile aplicate și verificarea fiecărei componente să aibă tipul necesar de "body" și scripturile necesare, s-a putut trece la setarea proprietăților brațului și setarea constrângerilor între componente.

6.4 Setarea proprietăților

Setarea proprietăților constă în alegerea valorilor pentru rigiditatea, amortizarea și forța pentru ca componentele să poată fi folosite. Realizarea acestor setări a fost necesară deoarece:

- Rigiditatea reprezintă forța cu care articulația ajunge la țintă. Dacă are valoarea de 0 sau valori mici de exemplu sub 10000, atunci articulația nu se mișcă, sau face mișcări lente.
- Amortizarea reprezintă proprietatea care ajută la diminuarea/oprirea tremurațiilor brațului. Dacă are valoarea de 0 sau valori mici, de exemplu sub 100, atunci la fiecare mișcare a articulației aceasta ar începe să tremure ducând la instabilitate și probleme la fizica robotului, cum ar fi desprinderea și împrăștierea componentelor brațului.
- Forța pe care motoarele/articulațiile o au nevoie pentru a le putea acționa. La valori mici, de exemplu sub 1000 motoarele brațului n-ar avea forța necesară să se miște și în consecință nu vor putea fi folosite.

Având în vedere cele mai de sus au fost alese următoarele valori pentru proprietăți:

- Rigiditatea 100000
- Amortizarea 1000
- Forța 10000

Pentru aplicarea acestor valori există 2 abordări principale: Prima abordare constă în aplicarea acestor valori direct în proprietățile articulației. Însă pentru a folosi această abordare există o problemă: valorile puteau să fie aplicate numai în momentul în care scena se afla în modul de "play". După ieșirea din modul de "play" valorile reveneau la cele inițiale. Astfel de fiecare dată când trebuia testată mișcarea unei componente sau ale unui set de componente trebuia aplicate valorile de fiecare dată.

A doua abordare a fost și cea aleasă și folosită în implementare și constă în crearea și folosirea unui script separat pentru a putea suprascrive mereu valorile. Astfel chiar dacă scena se află în modul de lucru, acest script se execută ultimul suprascrivând valorile din secțiunea de proprietăți ale articulației.

Acest script folosește următoarele funcții:

- "Drive()" care primește ca parametrii un obiect de tip "articulation body", și proprietățile de rigiditate, amortizare și forță. Acestea sunt accesate și aplicate folosind componenta din Unity de "xDrive"

- "ApplySettings()", care apelează funcția de "Drive()" care este aplicată pe fiecare componentă în parte
- "Start()" este funcția care se apelează automat când aplicația pornește în care se apelează funcția de "ApplySettings()" cu o întârziere de 0.1 secunde

6.4.1 Compensări cinematice

Compensările cinematice reprezintă condiționarea mișcării unor componente față de alte componente.

În mediul din VR compensările au fost necesare pentru că față de robotul real în care dacă un segment principal se deplasa, cum ar fi cel vertical atunci se deplasau și tije de susținere de pe o parte și alta a segmentului vertical, când se dădea comanda să se deplaseze același segment principal, tije rămâneau fixe. În cazul susținerii pompei, acesta fiind un copil al segmentului orizontal principal, acesta îi copia unghiul la care se afla.

Compensările cinematice realizate pot fi grupate în 2 categorii:

- simple, în care o componentă are unghiul de deplasare în același sens de componenta care îi condiționează mișcarea sau în sens opus acesteia
- compuse, în care unghiul de deplasare al unei componente este condiționate de minim 2 componente

Pentru a crea compensările cinematice a fost creat un script, care are implementat funcțiile: "NegativeCompensation()", "PositiveCompensation()", "FixedUpdate()".

Funcția de "NegativeCompensation()" c preia ca parametrii un "ArticulationBody" care reprezintă componenta părinte care dă unghiul de deplasare și un "ArticulationBody" care reprezintă componenta copil pe care este aplicat unghiul de la părinte. Se folosește de unghiul componentei părinte exprimată în grade, pe care o aplică cu semn opus asupra unghiului la care trebuie să ajungă componenta copil. Pentru a seta unghiul componentei copil este necesară accesarea componentei "target" care se află în categoria de "Drive" a lui "ArticulationBody"

Funcția de "PositiveCompensation()" care preia ca parametrii un "ArticulationBody" care reprezintă componenta părinte care dă unghiul de deplasare și un "ArticulationBody" care reprezintă componenta copil pe care este aplicat unghiul de la părinte. Funcția folosește de unghiul componentei părinte exprimată în grade, pe care o aplică cu același semn asupra unghiului la care trebuie să ajungă componenta copil. Pentru a seta unghiul componentei copil este necesară accesarea componentei "target" care se află în categoria de "Drive" a lui "ArticulationBody"

"FixedUpdate()" reprezintă funcția principală a script-ului și care față de metoda "Update()" care este apelată la fiecare cadru, este apelată de obicei la un interval de 0.02 secunde și este folosită pentru a se asigura că calculele legate de fizică rămân consistente. În această funcție sunt apelate funcțiile de mai sus, și sunt realizate restul calculelor pentru componentele ce necesită compensare față de 2 sau mai multe componente

Compensările au fost grupate în 2 categorii principale: pentru tije de susținere și pentru segmentele principale.

Pentru tije compensările au fost realizate astfel:

- "PositiveCompensation(segmentului vertical principal, tija verticală de susținere)"
- "PositiveCompensation(segmentul vertical principal, tija orizontală de susținere 2)"
- "NegativeCompensation(segmentul orizontal principal, tija orizontală de susținere 1)"
- tija verticală de susținere 1 se va deplasa în sens opus cu suma unghiurilor segmentul lateral și a segmentului vertical principal

În cadrul segmentelor principale a fost necesară setarea de constrângeri pentru segmentul orizontal principal deoarece când se deplasa segmentul vertical principal, acesta îi copia unghiul de deplasare, fiind necesar ca segmentul orizontal principal să-și păstreze poziția orizontală pe toată durata mișcării segmentul vertical principal. Tot în categoria segmentelor principale a fost necesară și setarea de constrângeri pe suportul de pompă deoarece indiferent de sensul de deplasare al segmentelor principale, acesta trebuie să rămână perfect paralelă cu solul.

Astfel pentru segmentul orizontal principal și suportul pompei, au fost setate următoarele constrângeri:

- segmentul orizontal principal se va deplasa în sens opus cu suma unghiurilor segmentului vertical principal și segmentul care deplasează segmentul orizontal principal pe axa Y
- suportul pompei are următoarea constrângere:

$$\begin{aligned} \text{suportPompei} = & (2 * \text{unghiulTijeOriZontale1}) \\ & - \text{unghiulSegmentuluiOriZontalPrincipal} \\ & - \text{unghiulSegmentuluiVerticalPrincipal} \end{aligned}$$

6.5 Sistemul de control al brațului în Unity

Sistemul de control în Unity reprezintă modul în care comenzile provenite de la manete câștii controlează componentele care sunt direct conectate la motoare. Fără acest sistem de control, robotul din Unity ar fi fost doar de decor. În implementarea sistemului de control au fost folosite 2 metode de implementare. Prima metodă se bazează pe controlul direct al motoarelor folosind butoanele de pe manete, cuprinzând și partea de resetare a poziției robotului, cât și funcționalitatea de a manevra obiecte. A doua metodă este mai utilizată și se bazează pe un algoritm de cinematică inversă și pe un efectuator final.

6.5.1 Metoda de control bazată pe control direct

Metoda aceasta este cea mai simplă și se bazează pe atribuirea modului de deplasare al componentelor părinte direct la butoanele manetei drepte. Pentru implementare s-a creat un script prin care se face această mapare.

Script-ul la bază implementează 2 funcții principale.

Prima funcție "OnEnable()", este cea care permite setarea și folosirea butoanelor manetelor folosite în control. Cea de a 2 a funcție este cea de "Update()" în care se realizează efectiv controlul. Însă pentru controlul efectiv a fost necesar folosirea proprietății "direction" a "joint-urilor", fiind accesată folosit componenta de "JointControl".

Pentru controlul zonei rotative, se citesc valorile de pe axa X a "thumbstick-ului", fiind-ui setat, praguri datorită sensibilității, acesta fiind foarte sensibil la atingere. Pentru rotirea spre dreapta dacă s-a depășit acel prag, direcția de deplasare va fi negativă, iar pentru stângă, tot dacă s-a depășit un prag, direcția de deplasare va fi pozitivă.

Pentru segmentul vertical principal, s-au folosit cele 2 "triggers" de pe manetă. Dacă se detectează o apăsare pe "trigger-ul" frontal atunci, sensul de deplasare este pe direcția negativă a axei Z, adică în față, iar dacă "trigger-ul" lateral, dacă se detectează o apăsare, sensul de deplasare este pe partea pozitivă a axei Z, adică în spate.

Pentru segmentul lateral de urcare-coborâre au fost folosite cele 2 butoane de pe manetă. Coborârea pompei se realizează pe direcția pozitivă a axei Y, la apăsarea butonului principal, iar urcarea pompei se realizează pe direcția negativă a axei Y, la apăsarea butonului secundar.

6.5.2 Metoda de control bazată pe cinematică inversă

Această metodă de control a unui braț robotic/robot este cea mai utilizată și se bazează pe implementarea unui algoritm ce rezolvă cinematica inversă. Algoritmul implementat pentru braț se bazează pe geometrie și trigonometrie și se folosește de legea cosinusului. Implementarea a fost realizată folosind 2 funcții într-un script dedicat. Prima funcție de "FixedUpdate()" este cea în care conține pașii de dinaintea implementării algoritmului și implementarea propriu-zisă a algoritmului. Cea de a 2 a funcție "SetJointAngle()" se ocupă de setarea unghiului "joint-urilor", din componenta de "xDrive" a unui "ArticulationBody".

În funcția "FixedUpdate()" pașii de dinaintea implementării a constat în setarea unor constante ce reprezintă lungimea segmentelor ale robotului și scalarea pe axele X, Y, Z a elementelor componente în raport cu dimensiunea la care se află robotul în scenă. Implementarea propriu-zisă a algoritmului a fost realizată prin a reduce mai întâi problema cinematicii inverse din planul 3D la planul 2D, formând un triunghi dreptunghic, pentru simplificarea calculelor ulterioare. Reducerea orizontală a fost realizată prin calculul distanței de la centrul bazei până la țintă pe orizontală, de unde s-a scăzut "offset" pentru segmentul de deplasare stânga-dreapta, acesta nefiind așezat perfect pe în centul axului care rotește brațul stânga-dreapta, reducerea fiind notată cu r :

$$r = \sqrt{x^2 + y^2} - L_1 \quad (1)$$

unde x și y reprezintă coordonatele în plan orizontal, și L_1 reprezintă un "offset" pentru segmentul de deplasare stânga-dreapta, acesta nefiind așezat perfect pe în centul axului care rotește brațul stânga-dreapta

Reducerea pe verticală a fost realizată prin scăderea din valoarea de pe axa Z a înălțimii bazei fixe, reducerea fiind notată cu h :

$$r = z - L_0 \quad (2)$$

unde z reprezintă coordonata în plan vertical, și L_0 reprezintă înălțimea bazei fixe

După realizarea calculelor, pe baza reducerilor a fost calculată distanța totală directă pentru țintă folosind reducerile calculate folosind formula (2) și

$$d = \sqrt{r^2 + h^2} \quad (3)$$

au fost setate 2 limite necesare: o limită în cazul în care brațul este complet întins, și cealaltă în cazul în care ținta este prea aproape de baza brațului, ceea ce ar duce la plierea brațului și posibilă la desprinderea componentelor. Pentru limite s-a un introdus un factor de scalare s care se calculează ca

suma dintre 2 laturi împărțite la distanța totală calculată. În cazul limitei de pliere a brațului, se recalculează factorul s ca raport dintre valoarea absolută dintre diferența celor 2 laturi la care s-a adunat o valoare de siguranță și distanța totală. În cazul în care se ating limitele se înmulțesc r, h cu factorul s , moment în care coordonatele țintei se ajustează proporțional cu factorul de depășire s , făcând ca brațul să rămână perfect întins și să nu se plieze peste limita admisă.

După setarea limitelor și pe baza calculelor de mai sus au fost calculate cosinusurile unghiurilor interioare pentru segmentul vertical principal și segmentul lateral de urcare-coborâre folosind legea cosinusului:

$$\cos A = \frac{L_2^2 + L_3^2 - d_2}{2 * L_2 * L_3} \quad (4)$$

și

$$\cos B = \frac{L_2^2 + d_2 - L_3^2}{2 * L_2 * d} \quad (5)$$

unde,

laturile L_2 și L_3 reprezintă tija de susținere prinsă între segmentul lateral și segmentul orizontal principal, respectiv segmentul vertical principal și d distanța totală

Pentru finalizarea calculelor și folosirea valorilor în calculul unghiurilor finale a fost necesară convertirea calculelor pentru cosinus la unghiurile necesare și calcularea unei noi valori ca pompa îndreptată direct în jos. Valoarea pentru pompă s-a notat cu c și s-a calculat folosind formula:

$$c = \arctan(h/r) \quad (6)$$

și pentru convertirea la unghiurile necesare a fost aplicată arccosinus care a fost convertită din radiani în grade folosind constanta definită în Unity de 57.29578, acestea fiind notate cu a , respectiv b . Astfel unghiurile pentru segmentul lateral și segmentul vertical principal au fost calculate prin:

$$\text{unghiSegmentVertical} = 90 - a \quad (7)$$

$$\text{unghiSegmentLateral} = -90 + (b + c) \quad (8)$$

Valorile de 90 reprezintă gradele în care cele 2 componente stau în starea de repaus inițială, mișcările produse de braț ducând la modificări pozitive sau negative din aceste valori. Aceste valori le-au fost setate semne diferite pentru a realiza mișcarea pe sensurile provenite de la efectorul final. Aplicările efective pe "joints" au fost realizate prin actualizări continue ale unghiurilor calculate mai sus folosind funcția de "Mathf.LerpAngle()" în care: se citește

unghiul curent la care se află brațul, se verifică unghiul la care trebuie să ajungă segmentul respectiv, și se calculează o valoare intermediară, și la final suprascrive unghiul vechi cu cel nou calculat.

Pentru completarea implementării trebuie și partea de efector final pe care brațul să o urmeze permițându-i astfel să poată fi controlat. Acest efector poate fi un obiect din scenă sau pompa care sunt controlate prin intermediul manetei. Pentru implementare a fost folosit ca efector un obiect sferic în scenă. Pompa sau suportul de pompă nu a putut fi folosit datorită că sunt copii ale altor componente, și folosirea lor putea duce la efecte neașteptate din partea brațului virtual.

Parte de efector final a fost implementată într-un script separat. Scriptul pentru efector implementează următoarele funcții: "OnEnable()" care permite folosirea butoanelor, și "Update()", care conține implementarea propriu-zisă.

În "Update()" se definește un vector ce conține 2 coordonatele pentru mișcarea efectorului: un punct pentru axa X și un punct pentru axa Y. După definire se verifică dacă cele 2 perechi de deplasare sus-jos, față-spate nu sunt nule. În cazul în care nu sunt nule în funcție de direcția de deplasare se incrementează sau decrementează din axe o valoare bazată pe viteza de deplasare a efectorului cu "Time.deltaTime" care permite deplasarea în scenă la o viteză constantă indiferent la câte cadre pe secundă rulează scena. De asemenea tot în această funcție se implementează partea de reset a efectorului la poziția inițială în scenă, poziția fiind dată de un vector ce cuprinde cele 3 coordonate din 3D. La final se folosește funcția "Mathf.Clamp", care preia poziția curentă a obiectului și o forțează să rămână strict într-un interval de valori, fiind necesar pentru ca în cazul în care efector se deplasează pe distanțe, componentele brațului care îl urmărește să nu provoace comportamente neașteptate. În acest script a mai fost necesară separarea mișcării de rotație a robotului față de efector, deoarece acesta se putea roti numai când brațul era întins, fiind folosită implementarea de la modul de control direct.

De asemenea partea de control mai implementează două funcționalități importante: partea de reset a brațului care poate fi folosită indiferent de modul de control și cea în care robotul să poată să manevreze obiecte.

6.6 Resetarea robotului

Resetarea brațului la poziția inițială este realizată printr-un script dedicat. Implementarea se folosește de următoarele funcții: "OnEnable()", "FixedUpdate()", și cea de "ResetPosition()".

Funcția de "ResetPosition()" preia ca argument un "ArticulationBody" căruia vrem să-i resetăm poziția. Resetarea poziției este realizată prin setarea proprietății "target" din "xDrive-ul" argumentului la valoarea de 0, aceasta fiind valoarea de bază la care se află "joints" ale lui "ArticulationBody". În funcția "FixedUpdate()" se realizează apeluri succesive ale "ResetPosition()" pe "ArticulationBodys" părinte folosire în compensări, numai dacă butonul atribuit resetului a fost apăsător.

6.7 Manevrarea obiectelor

Această funcționalitate este cea care permite brațului prin intermediul pompei să apuce obiecte și să-l lase. Se bazează pe următoarele funcții: "OnEnable()" necesară folosirii butoanelor de activare/dezactivare pompă și de lăsare a obiectului când brațul se resetează, "Update()" funcția principală a script-ului, "PickUpObject()" care se ocupă de prinderea obiectelor, "DropObject()", care se ocupă de desprinderea obiectului, "MoveObject()" care se ocupă de parte de deplasare al obiectului.

Această funcționalitate față de celelalte se folosește de componenta de "RayCast" și de "RigidBody", fiind aplicare pe obiectul pe care brațul trebuie să-l manipuleze.

În funcția "PickUpObject()" preia ca argument un obiectul care trebuie manipulat, verifică dacă este de tipul "RigidBody()" și dacă este de acel tip copiază obiectul de tip "GameObject" într-o altă variabilă de tip "RigidBody()", și pe acea variabilă sunt făcute următoarele setări:

- dezactivarea gravitației astfel încât în momentul când obiectul este apucat să nu fie tras de gravitația din Unity
- aplicarea unei constrângeri care prin care obiectul să nu se poată roti
- setarea părintelui obiectului ca fiind pompa
- activarea proprietății de "isKinematic" pentru ca obiectul să poată fi controlat de către sistemul de fizică, respectiv de către braț
- la final se atribuie obiectului global de tip "GameObject" argumentul funcției și implicit setările realizate

În funcția de "DropObject()" preia ca argument o variabilă de tip bool folosită ca să marcheze dacă obiectul să-și mențină sau nu poziția în care se află,

fiind necesară parte a sistemului de reset al brațului, și în cazul în care se activează reset-ul pompa să lase obiectul în ultima poziție în care se afla. Astfel se folosește o variabilă de tip Vector3 în care se salvează poziția curentă a obiectului, se verifică dacă obiectul curent nu este nul și dacă nu este setează astfel proprietățile de mai sus:

- reactivează gravitația obiectului
- dezactivează constrângerea
- setează părintele ca fiind nul
- dezactivează proprietatea de "isKinematic"
- obiectul manevrat și obiectul curent de tip "RigidBody" devin nule

De asemenea se verifică valoarea argumentului. Dacă este adevărată atunci poziția sa devine cea salvată în variabila de tip Vector, viteza liniară și cea unghiulară a obiectului sunt setate la 0. În caz contrar obiectului a ajuns la poziția dorită, și poziția și viteza liniară devin cele ale pompei.

Funcția de "MoveObject()", verifică mai întâi dacă distanța dintre obiect și poziția pompei s-a modificat. Dacă s-a modificat atunci se calculează direcția de deplasare a obiectului, care este de tipul Vector3, ca diferență între poziția pompei și poziția obiectului. La final este calculată forța obiectului necesară să ajungă la destinație ca produs între distanța de deplasare cu un factor de multiplicare ales arbitrar.

În funcția "Update()" se apelează funcțiile implementate mai sus pe baza unor condiții. Pentru funcția "DropObject()", apelul se realizează atunci când se apasă butonul de reset, moment în care obiectul manevrat își păstrează poziția sau dacă se apasă din nou butonul a fost apăsat a doua oară de la momentul când acesta a fost apăsat pentru a ridica un obiect. Funcția "PickUpObject()" se apelează în momentul în care s-a apăsat butonul pentru prima dată și dacă variabila "holdObject()" nu este nulă. Astfel ridicarea obiectului se realizează printr-un "RayCast" care pornește de la poziția pompei și se îndreaptă în direcția de jos a acesteia. Acest "RayCast" se duce până la o distanță maximă setată, detectând doar obiectele care fac parte dintr-un anumit strat setat, ridicarea realizându-se în momentul în care "RayCast" intersectează un obiect. Pentru "MoveObject()" este verificat dacă un obiect este ținut, și dacă este, funcția este apelată.

6.8 Sistemul de antrenare pentru operatorul uman

Pentru ca putea utilizatorul să poată să învețe să controleze brațul robotic s-a implementat o secțiune de tutorial. Acesta cuprinde o listă cu controalele brațului și cât o listă cu sarcini pe care să le execute utilizatorul pentru a învăța să folosească brațul.

6.8.1 Lista cu controale

Pentru lista cu controalele brațului a fost creat un JSON care cuprinde o structură de clase și vectori care cuprinde tipul butonului de pe manetă, care este butonul respectiv și ce acțiune realizează acest buton. Pentru citirea JSON-ului s-a creat un script nou ce conține 3 clase: clasa "JSONReader" care cuprinde partea de citire din JSON și afișarea conținutului acestuia în scenă, clasa "Controller" care cuprinde vectori de șiruri de caractere, ce conțin informații despre denumirea hardware a butonului și acțiunea asociată cu acesta din JSON și clasa "Controllers" ce cuprinde 2 vectori ce conțin elemente pentru maneta dreaptă, respectiv maneta stângă. Un aspect esențial pentru a putea citi din JSON este ca clasele "Controller" și "Controllers" să aibă atribuit atributul de "Serializable".

Pentru afișarea "JSON-ului" în scenă a fost folosit utilitarul TMPPro, și o componentă de tip TMP. Citirea "JSON-ului" se face în funcția "Start()" prin intermediul funcției "File.ReadAllText()" ce preia ca parametru un șir de caractere ce conține calea din sistem a fișierului JSON și returnează un șir de caractere cu conținutul fișierului.

Componenta "JSONUtility" din Unity preia conținutul obținut din funcția "File.ReadAllText()" îl deserealizează și realizează maparea conținutului pe structura celor 2 clase definite anterior. Toate datele citite sunt atribuite unei noi variabile de tip șir de caracter care la final este atribuită componentei TMP pentru a putea fi afișate în scenă.

6.8.2 Lista cu instrucțiuni de bază

Pentru partea de instrucțiuni de bază legată de funcționarea și controlul robotului s-a procedat în același mod: a fost creat un JSON cu acțiuni care trebuie executate, s-a creat un nou script care cuprinde clasa "Tutorial" care cuprinde vectori de șiruri de caractere, ce conțin informații despre acțiunea care trebuie executată, clasa "Tutorials" care cuprinde un vector de șiruri de caractere ce include elementele pentru tutorialul de bază, și "JSONScript" care cuprinde partea de citire din JSON și afișarea conținutului acestuia în scenă.

Însă pentru fiecare acțiune realizată s-a adăugat un indicator pentru a marca dacă acțiunea cerută a fost realizată sau nu. Acest indicator se bazează pe butoanele de scenă din Unity care își schimbă culoarea astfel: roșu dacă

acțiunea nu a fost realizată, verde dacă acțiunea a fost realizată. Pentru acest indicator a fost creat un script nou ce are în componență următoarele funcții: "OnEnable()", "OnDisable()", "initColor()" care preia ca culoarea efectivă care este aplicată, "setColorProperties()" care preia parametru indicatorului din Unity, blocul de culoare, și cel de culoare efectivă "setColor()" care pe lângă parametrii de indicator din scenă, bloc de culori și culoare efectivă preia și ca parametru butonul de pe manetă care este apăsat, "setColorThumbstick()" care în loc de buton, preia ca parametru "thumbstick-ul" care realizează acțiunea, "setPumpColor()" care preia ca parametru un butonul , 2 butoane din scenă, blocul de culoare și culoarea efectivă, funcția "Awake()" care setează culoarea de bază a butoanelor și "Update()" în care sunt apelate funcțiile legate de culori.

Funcția "initColor()" în care sunt inițializate componentele necesare manipulării indicatorilor, aceștia fiind: indicatorul folosit reprezentând o componentă de buton de Unity, blocul de culori pentru setarea culorilor pe componenta de buton, tipul de culoare care să fie aplicat, și aplicarea culorii pe indicatorul sau butonul din scenă respectiv.

Funcția de "setColorProperties()" este folosită pentru a seta proprietăți de culoare pe indicatoarele din Unity, acestea fiind:

- culoarea pe care o preia indicatoarele din Unity după cele de pe manete au fost apăstate, în acest caz fiind culoarea verde
- culoarea de evidență în momentul în care cursorul de la mouse se află peste acesta sau când maneta de VR este îndreptată către acesta fiind setată pe culoarea alb
- culoarea de tranziție a butonului între stările de neapăsat și apăsat fiind setată pe alb
- culoarea care marchează că butonul nu poate fi folosit fiind setată pe alb
- factorul de multiplicare al culorii care indică intensitatea culorii de pe indicatoare, este setată la valoarea 5 reprezentând o intensitate ridicată a culorii

La final deoarece modificările s-au realizat pe o copie a structurii culorilor se preia copia și se aplică pe indicatoarele din scena din Unity.

Funcția "setColor()" este folosită pentru a seta culoarea pentru acele indicatoare din Unity care reprezintă o acțiune executată prin intermediul butoanelor și "triggers" de pe manete. Aceasta verifică dacă butonul a fost apăsat, dacă a fost apăsat atunci este apelată funcția "setColorProperties()" cu parametrii necesari funcției.

Funcția "setColorThumbstick()" este folosită pentru a seta culoare pentru acele indicatoare din Unity care reprezintă o acțiune executată prin "Thumbsticks" ale manetelor. Butoanele de "Thumbsticks" au fost separate deoarece acestea sunt văzute în Unity ca 2 variabile diferite. Principiul de funcționare este același ca cel al funcției "setColor()".

Funcția "setPumpColor()" este folosită a reprezenta dacă acțiunea de activare sau dezactivare a pompei a fost realizată. Pentru reprezentarea acțiunilor de activare sau dezactivare în scenă au fost folosite 2 butoane. Prima dată se verifică dacă butonul a fost apăsat sau nu în cadrul curent față de celelalte funcții, unde se detecta o apăsare simplă. A fost folosit acest tip de verificare deoarece în Unity unui buton apăsat normal îi este detectată în același cadru din scenă mai multe apăsări, față de detecția în același cadru unde apăsarea este realizată o singură dată. Încă o necesitate a acestui tip de verificare este acela că pompa este activă la prima apăsare de buton și dezactivată la cea de a 2 a apăsare de buton.

Pentru a putea schimba culoarea pe indicatoarelor a fost folosit un contor care se incrementează în momentul când butonul de pe manetă a fost apăsat. La prima apăsare contorul devine 1 ceea ce înseamnă că pompa a fost activată și indicatorul specific își schimbă culoarea. La a 2 a apăsare se incrementează din nou contorul ceea ce înseamnă că butonul a fost din nou apăsat ceea ce înseamnă că pompa a fost dezactivată și indicatorul specific își schimbă culoarea. La final dacă au fost detectate mai mult de 2 apăsări se resetează contorul și indicatoarele își păstrează culoarea.

Funcția "Awake()" este folosită pentru a seta o culoare inițială pentru butoane. Aici este apelată funcția de "initColor()" care setează culoarea inițială pentru indicatoarele din scenă.

Funcția "Update()" este cea în care sunt setate culorile de realizare a activității. În aceasta sunt citite axele de pe "Thumbsticks", este verificată dacă pentru acestea a fost trecut un anumit prag, prag setat datorită sensibilității acestora, după verificare este apelată funcția de "setColorThumbstick()". Pentru restul butoanelor fiind apelată funcția "setColor()", respectiv "setPumpColor()".

Bibliografie momentană

- [1] 6 types of industrial robots. Robotec. 2026. URL: <https://robotec.org/blog/types-of-industrial-robot.html> (visited on 02/19/2026).
- [2] Articulation Body component reference. Unity. 2026. URL: <https://docs.unity3d.com/6000.3/Documentation/Manual/class-ArticulationBody.html> (visited on 04/25/2026).
- [3] ArticulationBody. Unity. 2026. URL: <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/ArticulationBody.html> (visited on 04/25/2026).
- [4] Ayah Hamad and Bochen Jia. "How Virtual Reality Technology Has Changed Our Lives: An Overview of the Current and Potential Applications and Limitations". In: International Journal of Environmental Research and Public Health 19.18 (2022). ISSN: 1660-4601. DOI: 10.3390/ijerph191811278. URL: <https://www.mdpi.com/1660-4601/19/18/11278>.
- [5] Hardware Basic Learnig. Hiwonder. 2026. URL: https://docs.hiwonder.com/projects/MaxArm/en/latest/docs/5.Hardware_Basic_Learnig_formatted.html (visited on 05/31/2026).
- [6] Hiwonder MaxArm Open Source Robot Arm Powered by ESP32 Support Python and Arduino Programming Inverse Kinematics Learning. Hiwonder. 2026. URL: <https://www.hiwonder.com/products/maxarm?variant=40008714092631> (visited on 04/25/2026).
- [7] Intro to ROS. CLEARPATH ROBOTICS. 2026. URL: <https://www.clearpathrobotics.com/assets/guides/kinetic/ros/Intro%20to%20the%20Robot%20operating%20System.html> (visited on 02/19/2026).
- [8] Meta Quest 3S. Meta. 2026. URL: <https://www.meta.com/quest/quest-3s/> (visited on 04/25/2026).
- [9] MQTT: The Standard for IoT Messaging. MQTT. 2024. URL: <https://mqtt.org/> (visited on 04/25/2026).

- [10] Killman Paul. "Raycast in Unity". In: (2024). DOI: 10 . 1109 / ICCAR . 2015.7166008.URL:<https://medium.com/@pkillman2000/raycast-in-unity-4c1895dc33d6> (visited on 05/28/2026).
- [11] Rigidbody. Unity. 2025. URL: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Rigidbody.html> (visited on 05/28/2026).
- [12] The History And Evolution Of Digital Twin Technology. Medium. 2023. URL: <https://medium.com/@sahilbhutada28/the-history-and-evolution-of-digital-twin-technology-6a0d97ee454b> (visited on 05/22/2026).
- [13] URDF-Importer. Unity-Technologies. 2022. URL: <https://github.com/Unity-Technologies/URDF-Importer> (visited on 04/25/2026).
- [14] What Are Industrial Robots? MABUCHI-MOTOR. 2026. URL: <https://solution.mabuchi-motor.com/en/blog/8> (visited on 02/19/2026).
- [15] What can Unity do? Unity. 2026. URL: <https://learn.unity.com/course/unity-for-artists-curricular-framework-resources/tutorial/what-can-unity-do#5fa1bdc0edbc2a01f0fae7a5> (visited on 02/18/2026).
- [16] What Is a Digital Twin? Intel. 2026. URL: <https://www.intel.com/content/www/us/en/learn/what-is-a-digital-twin.html> (visited on 05/28/2026).
- [17] What is a digital twin? IBM. 2025. URL: <https://www.ibm.com/think/topics/digital-twin> (visited on 05/25/2026).
- [18] What is Autodesk Fusion? Autodesk. 2026. URL: <https://www.autodesk.com/solutions/what-is-fusion-360> (visited on 04/25/2026).
- [19] What Is Inverse Kinematics? MathWorks. 2026. URL: <https://www.mathworks.com/discovery/inverse-kinematics.html> (visited on 05/25/2026).
- [20] What is Pub/Sub Messaging? AWS. 2026. URL: <https://aws.amazon.com/what-is/pub-sub-messaging/> (visited on 02/19/2026).
- [21] What is URDF (Unified Robot Description Format)? FOXGLOVE. 2026. URL:<https://foxglove.dev/robotics/urdf> (visited on 02/19/2026).
- [22] What is virtual reality (VR) and how does it work? TEAM VIEWER. 2026. URL: <https://www.teamviewer.com/en/solutions/use-cases/virtual-reality-vr/> (visited on 02/19/2026).
- [23] XR Interaction Toolkit. Unity. 2025. URL: <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.0/manual/index.html> (visited on 04/25/2026).